IoT Temperature and Humidity Monitoring System

1. Project Overview

- This project is an **IoT-based temperature and humidity monitoring system** using an **Arduino UNO WiFi Rev2** and **DHT22 sensor**.
- The system collects temperature and humidity data from the DHT22 sensor and transmits the measurements through an MQTT communication network. The data is published from the Arduino to the **HiveMQ Cloud MQTT Broker** using a secure TLS connection.
- **Node-RED** receives and processes the MQTT messages, visualizes the sensor data on a dashboard, and forwards the measurements to **MongoDB** for persistent storage.
- The complete system demonstrates an end-to-end IoT data pipeline from sensor data acquisition to database storage.
- The main focus of this project is to validate reliable sensor data acquisition, MQTT communication, secure data transmission, data processing, visualization, and database storage.

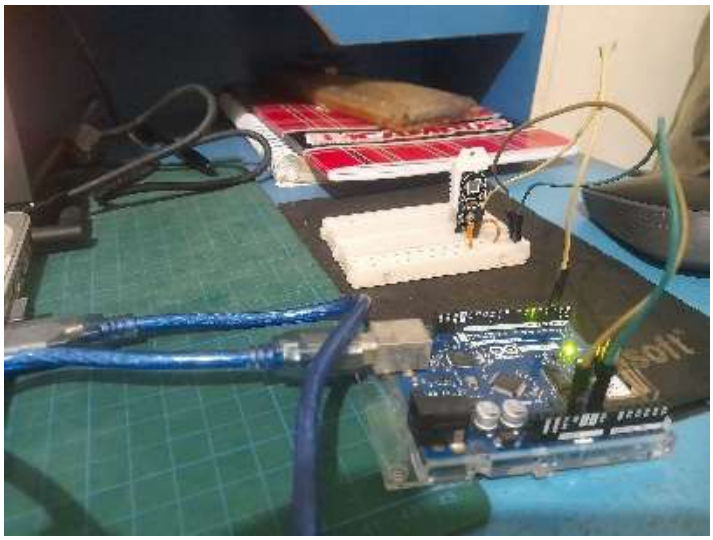


Figure 1. Arduino UNO WiFi Rev2 used as the IoT edge device

2. Problem Statement

Traditional sensor monitoring systems may only display data locally through a microcontroller or serial monitor. This can make remote data processing, visualization, and storage difficult.

This project adresses the following challenges:

1. How can temperature and humidity data be collected and transmitted through an IoT network?
2. How can sensor data be securely transmitted using MQTT and TLS?
3. How can MQTT data be received, processed, and visualized using Node-RED?
4. How can processed sensor data be stored in a database for monitoring and analysis?

The project therefore focuses on developing and validating a complete IoT data pipeline from sensor acquisition to database storage.

3. Objectives

The main objective of this project is to develop and validate a complete IoT data pipeline from the DHT22 sensor to MongoDB.

The specifics objective are:

1. Read temperature and humidity data from the DHT22 sensor.
2. Establish a stable Wi-Fi connection using the Arduino UNO WiFi Rev2.
3. Transmit sensor data using the MQTT protocol.
4. Establish a secure MQTT connection to HiveMQ Cloud using TLS.
5. Receive, process, and visualize MQTT messages using Node-RED.
6. Store sensor data in MongoDB.
7. Verify that sensor data is successfully transmitted, processed, visualized, and stored.
8. Document troubleshooting activities and system limitations encountered during implementation.

4. System Architecture

The system consists of five main stages:

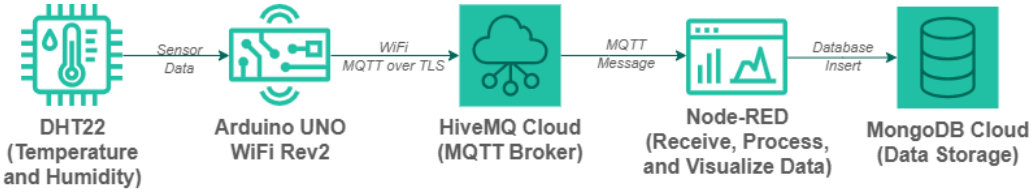


Figure 2. System Architecture of the IoT Monitoring System

Data Flow

1. The DHT22 measures temperature and humidity
2. The Arduino UNO WiFi Rev2 reads the sensor data.
3. The Arduino connects to the local Wi-Fi network.
4. The Arduino establishes a secure MQTT connection to HiveMQ Cloud.
5. Sensor data is published to an MQTT topic.
6. HiveMQ Cloud receives and distributes the MQTT message.
7. Node-RED suscribes to the MQTT topic.
8. Node-RED receives and process the payload.
9. Node-RED visualizes the processed data on a dashboard.
10. Node-RED sends the processed data to MongoDB.
11. MongoDB stores the sensor data for future retrieval and analysis.

5. Hardware

Table 1. Main Hardware Components

Component	Description
Arduino UNO WiFi Rev2	Main microcontroller and Wi-Fi communication device
DHT22	Temperature and humidity sensor
Breadboard	Prototype circuit assembly
Jumper Wires	Electrical connections
USB Cable Type-B	Programming and serial monitoring
PC/Laptop	Development, Node-RED, and database environment

DHT22

The DHT22 is used to measure temperature and relative humidity. The sensor provides digital output, allowing the Arduino to read measurement values through the DHT library.

The DHT22 is suitable for this prototype because it provides both temperature and humidity measurements using a single sensor.

6. Wiring Diagram

The DHT22 is connected to the Arduino UNO WiFi Rev2.

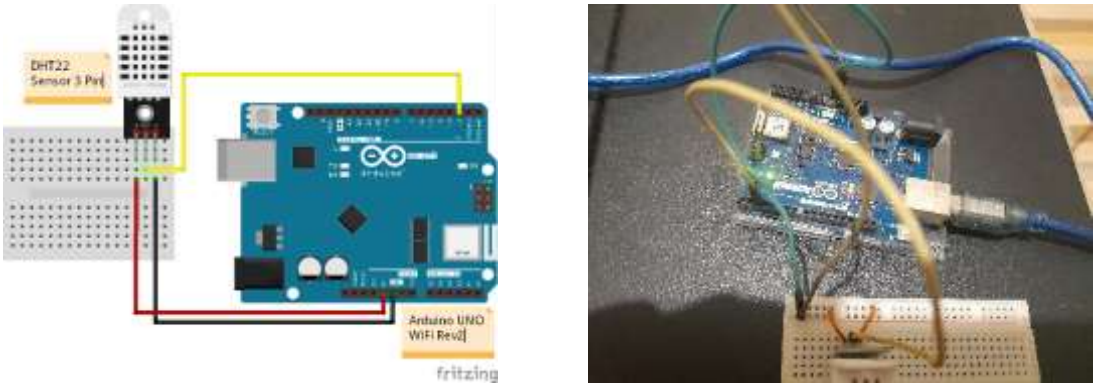


Figure 3. DHT22 wiring connection to the Arduino UNO WiFi Rev2

Table 2. DHT22 to Arduino Pin Connections

DHT22 Pin	Arduino UNO WiFi Rev2
VCC (+)	5V
DATA	Digital Pin 2 (D2)
GND (-)	GND

Wiring Considerations

- Ensure that VCC and GND are connected correctly.
- Ensure that the DATA pin matches the pin configured in Arduino firmware
- Avoid loose jumper-wire connections.
- Keep the sensor wiring short during testing where possible.

7. Software and Technologies

7.1. Arduino IDE

Used to:

- Develop the Arduino firmware.
- Configure Wi-Fi connectivity.
- Read DHT22 sensor data.
- Configure MQTT communication.
- Upload firmware to the Arduino Uno WiFi Rev2.
- Monitor system status through Serial Monitor

7.2. HiveMQ Cloud

Used as the MQTT broker between the Arduino and Node-RED.

The broker handles MQTT communication using:

- MQTT
- MQTT over TLS using port 8883
- Authentication
- MQTT topic-based communication

7.3. Node-RED

Used as the data-processing and visualization layer.

Node-RED is responsible for:

- Receiving MQTT messages.
- Processing the payload.
- Converting or formatting data when required.
- Visualising data using Node-RED Dashboard 2.0
- Forwarding the data to MongoDB

7.4. MongoDB

Used as the database layer for storing sensor measurements.

The database provides persistent storage so that sensor data can be retrieved and analyzed later.

8. Implementation

8.1. Firmware Implementation

The firmware was developed using the Arduino IDE for the Arduino UNO WiFi Rev2. The firmware is responsible for initializing the DHT22 sensor, establishing a Wi-Fi connection, connecting to the HiveMQ Cloud MQTT broker, reading temperature and humidity data, and publishing the measurements to the configured MQTT topics.

The following flowchart illustrates the main firmware execution sequence, from device initialization and network connection to sensor reading and MQTT data publishing.

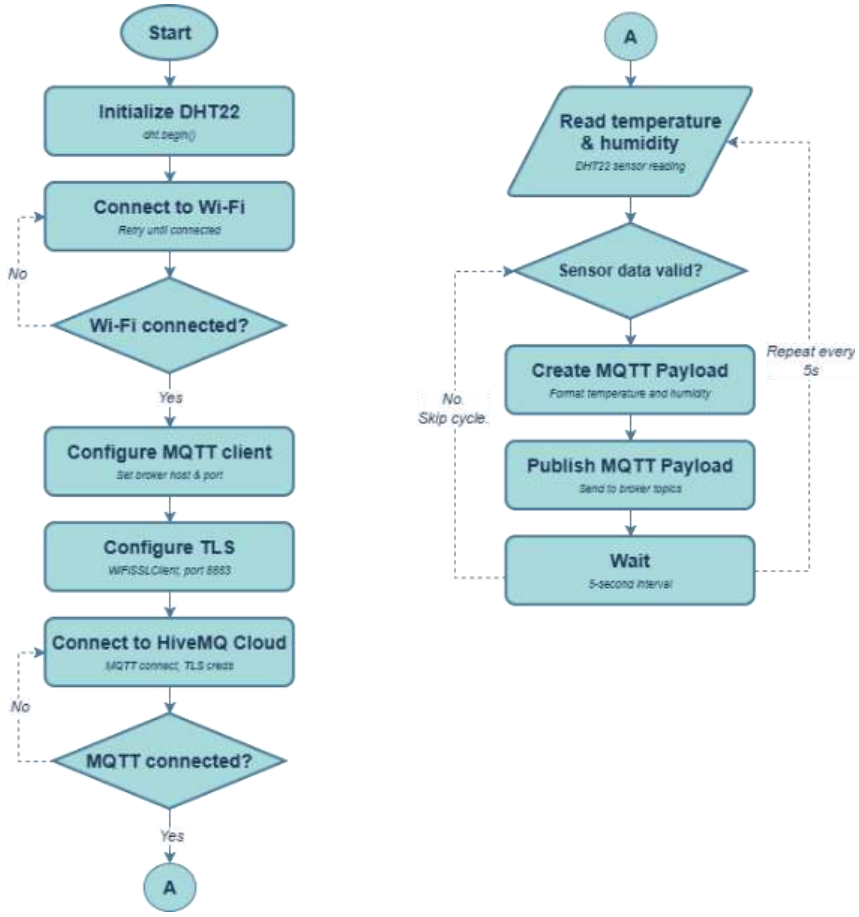


Figure 4. Arduino firmware execution flowchart

The Arduino firmware performs the following sequence:

1. Initialize the DHT22 sensor.
2. Initialize the Wi-Fi connection.
3. Verify the Wi-Fi connection, retrying automatically if disconnected.
4. Configure the MQTT client with the broker host and port.
5. Configure the secure TLS connection.
6. Connect to HiveMQ Cloud using the provided credentials.
7. Read temperature and humidity data from the sensor.
8. Create an MQTT payload from the sensor data.
9. Publish the payload to the configured MQTT topics.
10. Wait, then repeat the process periodically

The Arduino Serial Monitor was used during implementation to observe the firmware execution and verify the device status. The output provides information about Wi-Fi connectivity, MQTT connection status, sensor readings, and MQTT publishing activity.

```
11:26:18.606 ->
11:26:18.906 -> ================================
11:26:18.938 -> Arduino WiFi Rev2 + DHT22
11:26:18.970 -> HiveMQ Cloud MQTT TLS
11:26:18.970 -> ================================
11:26:19.004 -> [1] Starting Wi-Fi...
11:26:19.037 -> Wi-Fi: Connecting....
11:26:25.594 -> [2] Wi-Fi connected.
11:26:25.713 -> [3] Waiting 3 seconds...
11:26:28.566 -> [4] Starting MQTT connection...
11:26:31.012 -> [MQTT] Connected!
11:26:31.056 -> [5] Setup completed.
```

Figure 5. Arduino Serial Monitor showing successful Wi-Fi and MQTT connection.

```
11:26:31.076 -> Data Sensor:
11:26:31.076 -> Temperature: 32.3 °C
11:26:31.108 -> Humidity: 71.5 %
11:26:31.140 -> MQTT:
11:26:31.140 -> Temperature published
11:26:31.140 -> Humidity published
11:26:36.009 ->
11:26:36.009 -> Data Sensor:
11:26:36.009 -> Temperature: 32.3 °C
11:26:36.157 -> Humidity: 71.4 %
11:26:36.157 -> MQTT:
11:26:36.161 -> Temperature published
11:26:36.161 -> Humidity published
11:26:40.966 ->
11:26:40.966 -> Data Sensor:
11:26:40.966 -> Temperature: 32.3 °C
11:26:40.998 -> Humidity: 71.3 %
11:26:41.030 -> MQTT:
11:26:41.030 -> Temperature published
11:26:41.067 -> Humidity published
```

Figure 6. Arduino Serial Monitor showing DHT22 readings and MQTT publishing activity.

8.2. MQTT Implementation

MQTT was implemented as the communication protocol between the Arduino and HiveMQ Cloud. The firmware publishes temperature and humidity measurements using two separate MQTT topics.

Table 3. MQTT Topics and Payload Examples

Measurement	MQTT Topic	Payload Example
Temperature	DHT22/environment/temperature	32.8
Humidity	DHT22/environment/humidity	79.8

Each measurement is published as a numeric payload to its corresponding MQTT topic. The firmware publishes sensor data at a configured interval of **5 seconds**.

8.3. Node-RED Implementation

Node-RED is configured to perform the following functions:

1. Connect to HiveMQ Cloud
2. Subscribe to the MQTT topic.
3. Receive the sensor payload.
4. Convert and process the received numeric payloads.
5. Process the temperature and humidity values.
6. Prepare the data for MongoDB
7. Insert the data into the database.
8. Visualize data in Node-RED Dashboard 2.0

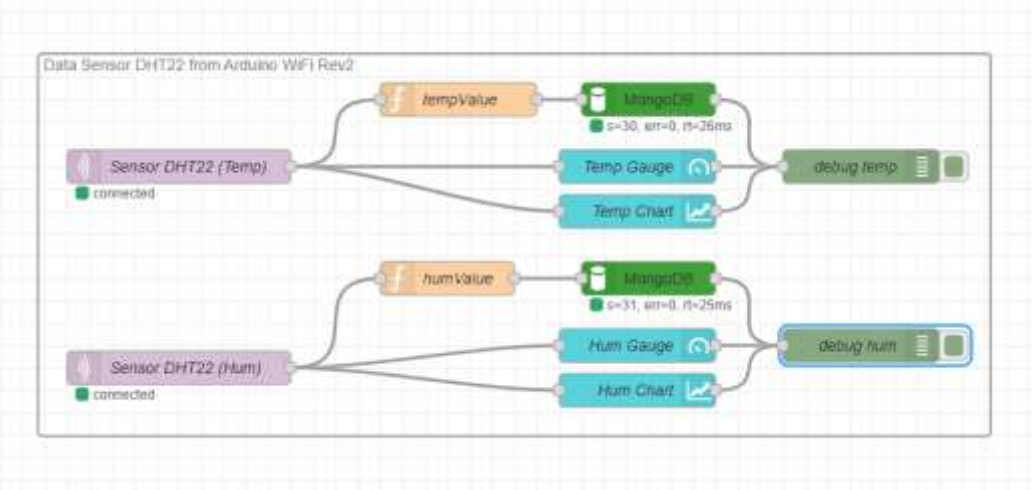


Figure 7. Node-RED flow for receiving, processing, visualizing, and storing MQTT sensor data.

Node-RED Dashboard 2.0 was configured to provide real-time visualization of the temperature and humidity measurements. Gauge components are used to display the current values, while chart components provide a visual representation of the sensor readings over time.

Figure 8. Node-RED Dashboard displaying real-time temperature and humidity measurements.

8.4. MongoDB Implementation

MongoDB is used as the persistent storage layer for the sensor data processed by Node-RED. Each sensor measurement is stored as an individual document. Temperature and humidity measurements are stored separately, consistent with the two MQTT topics used earlier in the data pipeline.

Figure 9. MongoDB collection containing stored temperature and humidity measurements.

Each document contains a timestamp associated with the stored measurement, allowing the sensor readings to be queried and analyzed chronologically.

9. MQTT Architecture

MQTT is used as the communication protocol between the Arduino UNO WiFi Rev2 and Node-RED through HiveMQ Cloud.

- **Arduino UNO WiFi Rev2 (*Publisher*):** Publishes temperature and humidity measurements.

- **HiveMQ Cloud (*Broker*):** Receives and routes MQTT messages to subscribed clients.
- **Node-RED (*Subscriber*):** Receives and processes MQTT messages from the configured topics.
- **Security:** MQTT communication uses TLS.
- **Publishing interval:** 5 seconds.

The system publishes sensor readings to HiveMQ Cloud using two separate topics, each carrying a plain numeric value:

DHT22/environment/temperature → e.g. 32.8

DHT22/environment/humidity → e.g. 79.8

The MQTT messages were verified using the HiveMQ Cloud Web Client.

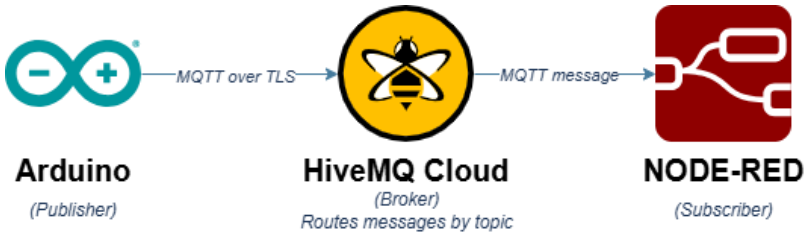


Figure 10. MQTT communication architecture between Arduino, HiveMQ Cloud, and Node-RED

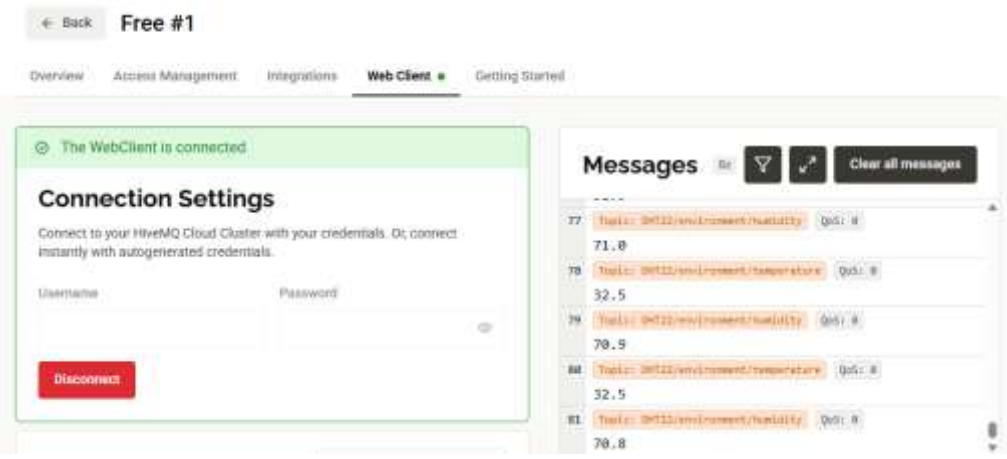


Figure 11. MQTT Payload Verification in HiveMQ WebClient

10. Testing & Validation

10.1. Wi-Fi Connection

Objective: Verify that the Arduino UNO WiFi Rev2 can connect to the configured Wi-Fi network.

Expected Result: The Arduino should successfully connect to Wi-Fi and obtain an IP address.

```
11:26:18.906 -> ================================
11:26:18.938 -> Arduino WiFi Rev2 + DHT22
11:26:18.970 -> HiveMQ Cloud MQTT TLS
11:26:18.970 -> ================================
11:26:19.004 -> [1] Starting Wi-Fi...
11:26:19.037 -> Wi-Fi: Connecting....
11:26:25.594 -> [2] Wi-Fi connected.
```

Figure 12. Arduino successfully connected to the Wi-Fi network.

Actual Result: The Arduino successfully connected to the Wi-Fi network and obtained an IP address.

Status: PASS

10.2. MQTT Connection to HiveMQ Cloud

Objective: Verify that the Arduino can establish an MQTT connection with HiveMQ Cloud.

Expected Result: The Arduino should successfully establish a secure MQTT connection using the configured HiveMQ Cloud and TLS settings.

```
11:26:25.594 -> [2] Wi-Fi connected.
11:26:25.713 -> [3] Waiting 3 seconds...
11:26:28.566 -> [4] Starting MQTT connection...
11:26:31.012 -> [MQTT] Connected!
11:26:31.056 -> [5] Setup completed.
```

Figure 13. Arduino MQTT successful connection to HiveMQ Cloud

Actual Result: The Arduino successfully established an MQTT connection to HiveMQ Cloud.

Status: PASS

10.3. DHT22 Sensor Reading

Testing and validation were performed to verify that each stage of the IoT data pipeline operated as expected. The tests covered sensor data acquisition, Wi-Fi connectivity, MQTT communication with HiveMQ Cloud, Node-RED data processing and visualization, MongoDB data storage, and the complete end-to-end data flow.

Objective: Verify that the Arduino can successfully read temperature and humidity data from the DHT22 sensor.

Expected Result: The Arduino should continuously obtain valid temperature and humidity readings from the DHT22.

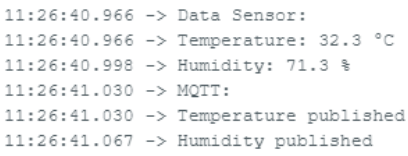


Figure 14. Arduino Serial Monitor showing DHT22 sensor readings.

Actual Result: The Arduino successfully read temperature and humidity values from the DHT22.

Status: PASS

10.4. MQTT Publishing

Objective: Verify that the Arduino successfully publishes temperature and humidity data to the configured MQTT topics.

Expected Result: The two MQTT topics should receive new messages periodically according to the configured publish interval.

Configured Topics:

- DHT22/environment/temperature
- DHT22/environment/humidity

Example Payloads:

- DHT22/environment/temperature → 32.8
- DHT22/environment/humidity → 79.8

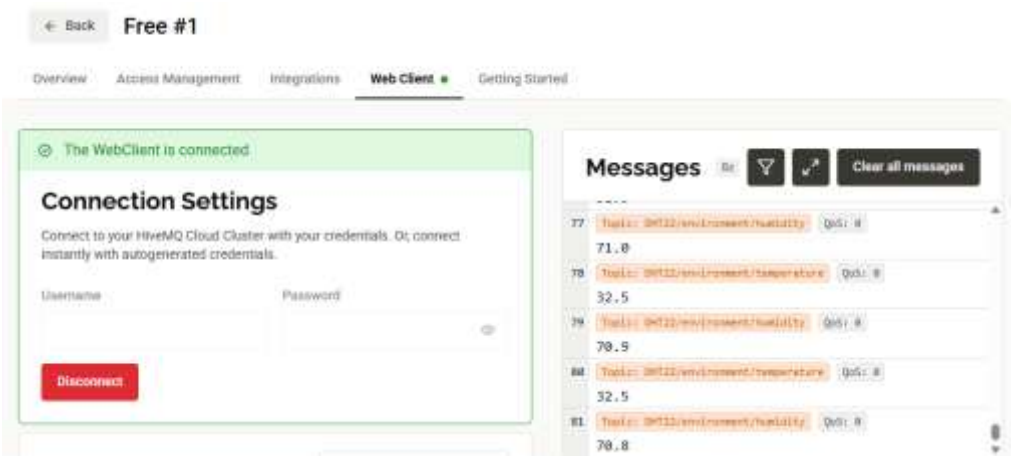


Figure 15. HiveMQ Cloud Web Client showing temperature and humidity MQTT messages.

Actual Result: Both topics successfully received sensor messages at the configured 5-second publishing interval.

Status: PASS

10.5. Node-RED MQTT Reception and Processing

Objective:

Verify that Node-RED successfully receives and processes the MQTT messages from HiveMQ Cloud.

Expected Result:

Node-RED should receive the published temperature and humidity messages and process them according to the configured flow.

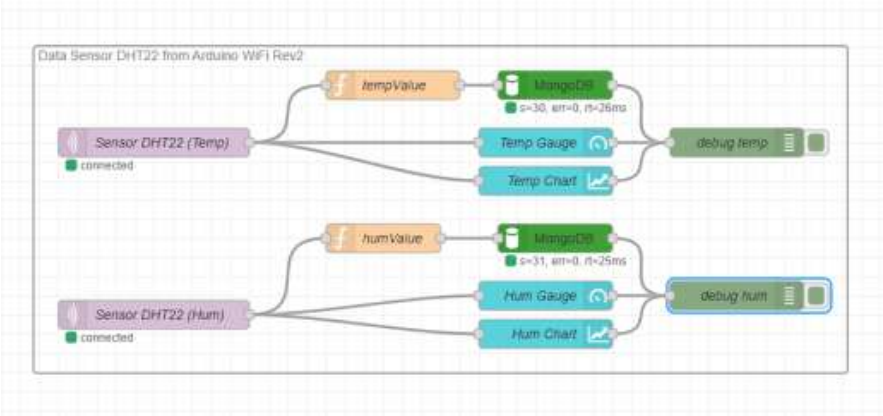


Figure 16. Node-RED flow for receiving and processing MQTT sensor data.

Figure 17. Node-RED debug displaying temperature and humidity data

Actual Result: Node-RED successfully received and processed the MQTT sensor data.

Status: PASS

10.6. Dashboard Visualization

Objective: Verify that the processed sensor data is correctly visualized on the Node-RED Dashboard.

Expected Result: The dashboard should display the current temperature and humidity values using the configured gauge and chart components.

Figure 18. Node-RED Dashboard displaying temperature and humidity data.

Actual Result: Temperature and humidity data were successfully displayed on the Node-RED Dashboard.

Status: PASS

10.7. MongoDB Data Storage

Objective: Verify that the sensor data received and processed by Node-RED is successfully stored in MongoDB.

Expected Result: New sensor measurements should appear as documents in the configured MongoDB collection.

Figure 19. MongoDB collection containing stored sensor measurements.

Actual Result: Sensor measurements were successfully stored in MongoDB.

Status: PASS

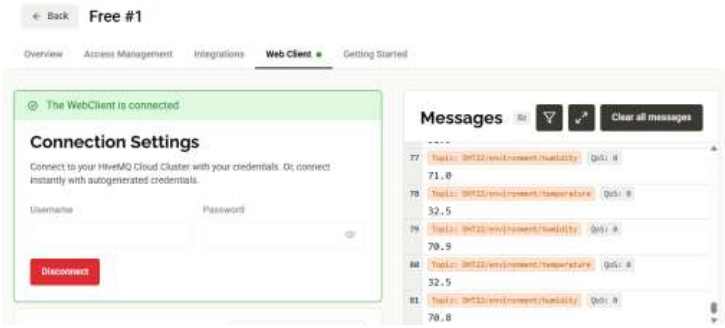
10.8. End-to-End Validation

Objective: Verify that sensor data can successfully travel through the complete IoT pipeline from data acquisition to database storage.

Expected Result: A sensor measurement generated by the DHT22 should be successfully transmitted through the complete pipeline, processed by Node-RED, visualized on the dashboard, and stored in MongoDB.

1. Arduino Serial Monitor — sensor reading, Wi-Fi, and MQTT connected.

2. HiveMQ Cloud Web Client — MQTT message.



3. Node-RED — received/processed message.

4. Node-RED Dashboard — visualized value.

5. MongoDB — stored document.

Figure 20. End-to-end validation evidence showing the complete IoT data pipeline.

Actual Result: The sensor data successfully passed through the complete IoT pipeline from the DHT22 to MongoDB.

Status: PASS

Each test was evaluated by comparing the expected result with the actual system behavior. Screenshots and system outputs are provided as evidence of the test results.

Table 4. Testing and Validation Summary

Test ID	Test	Expected Result	Actual Result	Status
T01	Wi-Fi connection	Arduino obtains IP	IP obtained	PASS
T02	MQTT connection	Arduino connects to HiveMQ	Connected	PASS
T03	DHT22 reading	Temperature & humidity displayed	Data displayed	PASS
T04	MQTT publishing	Data published every 5 seconds	Data received	PASS
T05	Node-RED reception	MQTT payload received	Payload received	PASS
T06	Dashboard	Sensor values displayed	Values displayed	PASS
T07	MongoDB insertion	Data stored as document	Documents stored	PASS
T08	End-to-end pipeline	Data reaches database	Data stored successfully	PASS

10.9. Validation Summary

The testing results demonstrate that the main components of the IoT system operated successfully. The DHT22 data was acquired by the Arduino, transmitted through MQTT to HiveMQ Cloud, received and processed by Node-RED, visualized on the dashboard, and stored in MongoDB. The complete end-to-end data pipeline was therefore successfully validated.

11. Troubleshooting

Several issues were encountered during the development and integration of the IoT communication pipeline. Troubleshooting was performed by checking the configuration and communication status of each layer, starting from Wi-Fi connectivity, MQTT communication, broker configuration, TLS security, Node-RED integration, and MongoDB connectivity.

The main troubleshooting cases encountered during the project are described below.

11.1. Wi-Fi Connection Issue

Problem: The Arduino initially experienced difficulties establishing a stable Wi-Fi connection.

Investigation: The Wi-Fi network configuration and credentials were checked. The firmware configuration was also reviewed to ensure that the correct network parameters were being used.

Solution: The Wi-Fi configuration and related firmware settings were corrected.

Result: The Arduino successfully connected to the Wi-Fi network and obtained network connectivity.

11.2. MQTT Connection Issue

Problem: The Arduino initially failed to establish the expected MQTT communication with the broker.

Investigation: The MQTT broker configuration, connection parameters, and client settings were reviewed. The broker address, port, authentication parameters, and MQTT client configuration were checked.

Solution: The MQTT configuration was updated and the client parameters were corrected to match the selected broker configuration.

Result: The Arduino successfully established an MQTT connection and was able to publish sensor data.

11.3. Broker Migration: Mosquitto to HiveMQ Cloud

Problem: The initial implementation used a local Mosquitto MQTT broker. During the integration process, the MQTT communication did not work as expected, which affected the intended communication between the Arduino and the MQTT processing layer.

Investigation: The MQTT broker configuration and client communication were reviewed. The project was initially tested using a local Mosquitto broker before the MQTT architecture was changed to use a cloud-based broker.

Solution: The MQTT broker was migrated from the local Mosquitto broker to HiveMQ Cloud. The Arduino MQTT client and Node-RED MQTT configuration were updated to use the HiveMQ Cloud broker.

Result: The Arduino successfully connected to HiveMQ Cloud and published sensor measurements through MQTT.

11.4. TLS Configuration Issue

Problem: After migrating to HiveMQ Cloud, the MQTT connection required secure communication using TLS.

Investigation: The MQTT client configuration was reviewed to identify the required secure broker parameters, including the broker hostname, secure MQTT port, authentication credentials, and TLS configuration.

Solution: The MQTT client was configured with the required TLS parameters and authentication settings for HiveMQ Cloud.

Result: The Arduino successfully established a secure MQTT connection to HiveMQ Cloud.

11.5. Node-RED

Problem: After migrating the MQTT broker to HiveMQ Cloud, Node-RED was still configured to use the previous broker configuration. As a result, Node-RED could not receive the sensor messages from the new MQTT broker.

Investigation: The Node-RED MQTT node configuration was checked. The broker settings and TLS parameters were compared with the configuration used by the Arduino.

Solution: The Node-RED MQTT configuration was updated to use the HiveMQ Cloud broker and the corresponding TLS settings.

Result:

Node-RED successfully connected to HiveMQ Cloud and received the published sensor data.

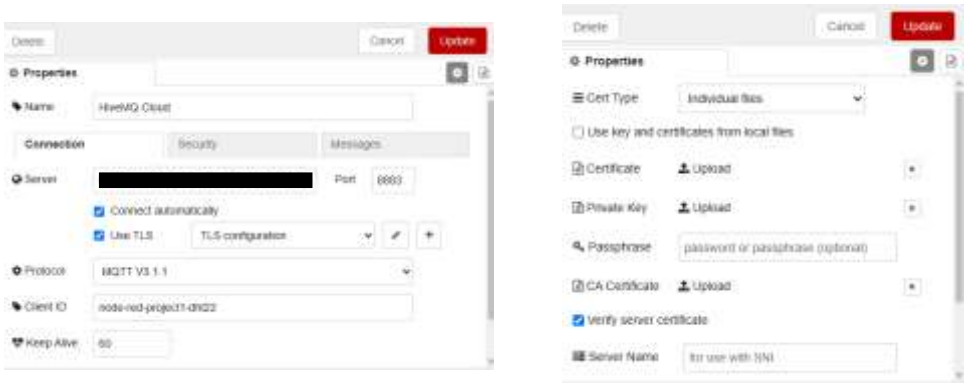


Figure 21. Updated Node-RED MQTT configuration and successful message reception.

11.6. MongoDB Integration Issue

Problem: MongoDB required additional configuration before Node-RED could successfully store the processed sensor data.

Investigation: The Node-RED database connection and MongoDB configuration were reviewed to ensure that the database and collection settings matched the intended storage structure.

Solution: The Node-RED MongoDB connection was configured and connected to the target database and collection.

Result: Node-RED successfully inserted the processed sensor measurements into MongoDB.

Figure 22. MongoDB integration and successfully stored sensor measurements.

12. Problems & Solutions

Table 5. Troubleshooting Summary

Problem	Cause/Investigation	Solution	Result
Wi-Fi connection issue	Network and credential configuration	Corrected Wi-Fi configuration	Arduino connected successfully
MQTT connection issue	Broker/client configuration	Updated broker and MQTT parameter	MQTT connection established
Broker migration	Initial local Mosquitto implementation	Migrated to HiveMQ Cloud	HiveMQ Cloud connection successful
TLS configuration	Secure MQTT connection required	Configured TLS parameters	Secure MQTT communication established using port 8883

Node-RED connection	Previous broker configuration remained	Updated MQTT broker and TLS settings	Node-RED received data
MongoDB Cloud integration	Database connection required additional configuration	Connected Node-RED to MongoDB Cloud	Sensor data stored successfully

13. Limitations

This project is a prototype designed to validate the IoT data pipeline.

The current limitations include:

1. The DHT22 is a low-cost sensor and its measurement accuracy is limited by the sensor specification and environment conditions.
2. The project does not perform formal sensor calibration.
3. The system has only been tested with a single DHT22 sensor.
4. The system depends on Wi-Fi network availability.
5. Loss of network connectivity can interrupt real-time data transmission.
6. The current system focuses on data acquisition, transmission, processing, visualization, analyzing, and storage.
7. MongoDB storage can grow continuously if the system publishes data at a high frequency without a data-retention strategy.
8. The prototype does not yet implement advanced fault handling such as automatic alerting when the sensor becomes unavailable.

14. Future Improvements

Several improvements can be implemented in future versions.

Hardware Improvements

- Improve enclosure and physical installation.
- Use higher-accuracy or industrial-grade sensors.
- Add additional sensor nodes.

Firmware / Device Improvements

- Add automatic MQTT reconnection handling.
- Add DHT22 sensor error detection.
- Add watchdog functionality.
- Implement configurable sampling intervals.
- Improve local fault handling.

Data Processing / Backend Improvements

- Improve MQTT payload validation.
- Add data integrity checks.
- Add MongoDB data retention policies.
- Implement historical data management.

Monitoring & Alerting Improvements

- Improve Node-RED Dashboard visualization.
- Add historical trend visualization.
- Add threshold-based alerts.
- Add notification systems for abnormal conditions.
- Integrate Grafana or another monitoring platform if required.

15. Conclusion

- This project successfully demonstrated an end-to-end IoT temperature and humidity monitoring system using the **Arduino UNO WiFi Rev2, DHT22, HiveMQ Cloud, Node-RED, and MongoDB**.
- The system successfully acquired temperature and humidity measurements, transmitted the data through **MQTT over TLS**, processed and visualized the data using **Node-RED**, and stored the measurements in **MongoDB**.
- The complete IoT data pipeline was successfully validated:
  DHT22 → Arduino UNO WiFi Rev2 → Wi-Fi → MQTT/TLS → HiveMQ Cloud → Node-RED → MongoDB
- The project also provided practical experience in MQTT communication, secure MQTT/TLS configuration, Node-RED integration, database connectivity, troubleshooting, and end-to-end system validation.